

# 2DX3 Final Report

**By:** Abdelrhman Aldeeb

**Date:** 2026-04-05

As a future member of the engineering profession, the student is responsible for performing the required work in an honest manner, without plagiarism and cheating.

Submitting this work with my name and student number is a statement and understanding that this work is our own and adheres to the Academic Integrity Policy of McMaster University and the Code of Conduct of the Professional Engineers of Ontario.

## Device Overview - Features

The system uses a Time-of-Flight (ToF) sensor that is mounted on a stepper motor to measure distances of a surrounding environment, and then create a 3D graph from the data acquired. The following specifications for the system are implemented:

- **Bus Speed:** The microcontroller operates at 28MHz
- **Operating Voltage:** Stepper motor, ToF sensor, on board buttons operate at 5-12VDC, 2.6-3.5V and 3.3V, respectively
- **Cost:** overall system is low cost with the ToF sensor (\$30), stepper motor (\$10) and finally the MSP432E4 (\$70), with the total cost being around \$110.
- **Serial Communication:** The system uses two communication protocols: UART and I2C. UART communication between the board and the computer is used to send the collected data from the microcontroller to the computer for visualisation. The UART operates at a baud rate of 115200 bps. I2C communication between the ToF sensor and the microcontroller for data.
- **Programming Language:** MATLAB is used for data acquisition, processing and visualisation
- **Memory:** The microcontroller uses on-chip flash memory to store the program and SRAM for buffers and runtime variables. The microcontroller has 1024KB of flash memory and 256KB of SRAM.

### Additional Features:

- 64 measurements in a full 360-degree rotation. Every 5.625 degrees, a measurement is taken
- Clockwise and counterclockwise motor rotation
- Push the button to control the motor start and stop
- LED indicators for measurement, UART status
- Additional LED indicator for when one 360 scan is completed

## Device Overview- General Description

The system scans a nearby environment in the y-z plane and creates a 3D plot based on the data. The ToF takes a measurement every 5.625 degrees for a total of 64 scans in a full rotation. The motor and ToF sensor begin operation when PJ0 is pressed and can be stopped when PJ0 is pressed again. LEDs PN0 and PN1 are flashed when a measurement is taken and at the start of transmission. After a full 360 scan, the motor is stopped for 5 seconds, and PF4 begins to flash. The motor then rotates in the opposite direction, and scanning is resumed. Matlab compensates for this by reversing the angle sequence when processing the data received.

## Signal Acquisition and ADC

The ToF sensor internally does the following:

- Shoots a laser pulse
- Detects the reflection
- Signal detection
- Analog to digital conversion

The output is a value in millimetres, which the microcontroller receives.

**Communication with PC**

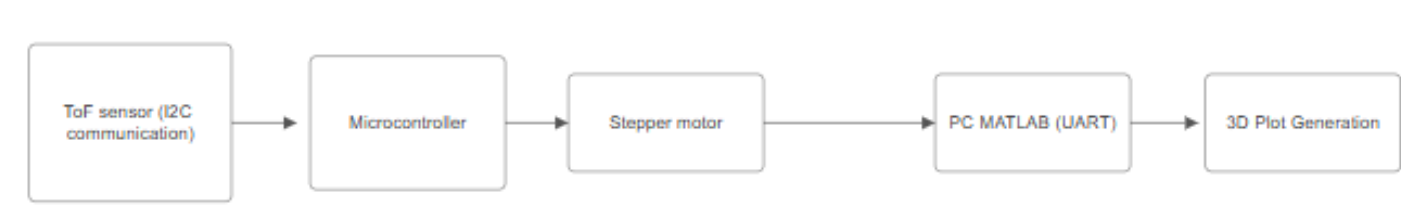
The microcontroller communicates with the computer using UART communication, and the baud rate is 115200 for data transmission.

**Visualization**

MATLAB is used to read the distance and status value. It checks if the status value is 0 (valid) and then converts the distance data into Cartesian coordinates. It then generates a 3D plot based on those coordinates.

**Block Diagram**

**Figure 1: Block Diagram**



**Table 1: Device Characteristics Table**

| Component     | Pins Used          | Details                                                                          |
|---------------|--------------------|----------------------------------------------------------------------------------|
| Tof Sensor    | SDA(PB3), SCL(PB2) | I2C communication                                                                |
| Stepper Motor | PH0-PH3            | Motor control                                                                    |
| Push Buttons  | PJ0                | Start/Stop                                                                       |
| LEDs          | PF0, PN0, PN1      | Status indication, and an additional LED for when a 360-degree scan is completed |
| Bus speed     | 28 MHz             | Assigned bus speed                                                               |
| UART          | COM4, 115200 bps   | Serial communication                                                             |

## Distance Measurement

The ToF sensor measures distance by emitting a laser and calculating the time it takes from the moment it is emitted to the moment it returns. The distance is calculated by:  $d = (c \cdot t) / 2$ . The distance is provided in millimetres. The microcontroller and ToF sensor communicate through the I2C protocol, where the microcontroller is the master device. In a full rotation, the system does 64 measurements, which is every 5.625 degrees. The conversion to Cartesian coordinates happens in MATLAB after receiving the data. The calculations are as follows:  $y = d \cdot \sin(\theta)$ ,  $z = d \cdot \cos(\theta)$ ,  $x = (\text{scan} - 1) \cdot (\text{x displacement per scan})$ . The y-z plane represents each 360-degree scan, where y is the height of the surrounding environment, and z is the width of the surrounding environment. The x-axis represents the displacement between scans.

### Example:

- scan=2
- x distance between scans=0.3m
- distance=2m
- Angle=30°

$$y = 2 \cdot \sin(30) = 1.0\text{m}$$

$$z = 2 \cdot \cos(30) = 1.73\text{m}$$

$$x = (2 - 1) \cdot (0.3\text{m}) = 0.3\text{m}$$

This produces the following coordinate:  $(x, y, z) = (0.3, 1.0, 1.73)$

## Displacement Implementation

Displacement is implemented manually by moving the system between scans. Each scan is spaced by a user-set distance, where the default is 0.3m. This is represented in MATLAB as  $x = (\text{scan} - 1) \cdot 0.3$ .

## Visualization

MATLAB completed the visualisation process using `serialport()` to establish a connection, `readline()` to read the data and `plot3()` to generate the 3D plot from the data. Each scan formed a 2D slice, which the multiple scans combined to make a 3D plot. Invalid points with non 0 status used the previous valid distance measurement, and then y and z are calculated using the formulas mentioned above.

## Application Example

The system can be used to scan indoor environments such as rooms or hallways, and then generate a 3D plot from the data.

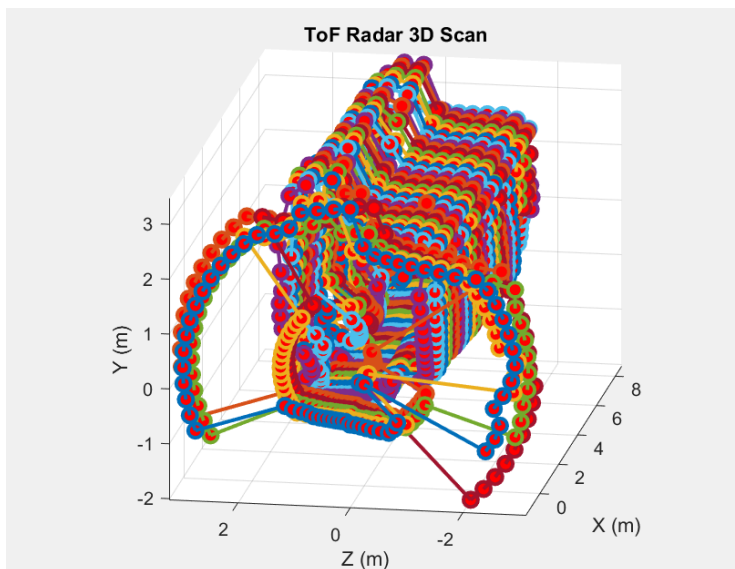
### Steps:

1. Connect the microcontroller to the PC
2. Identify the COM port by searching 'Device Manager' in the Windows search bar. Find Ports and press on the arrow. Find the number in Application/User UART(COM #). Take note of the number as it is needed in the next step
3. Open MATLAB and enter the port number in line 24 of the program
4. Go to line 47 in MATLAB to change to the number of scans you want to take (default 3). Then go to line 75 to change X displacement per scan
5. Go to line 38 in Keil to change the number of scans (must match the MATLAB number)
6. Build the circuit as listed in **Table 1**
7. Flash the microcontroller code by going to Keil and clicking translate, build and finally load. Then hit the reset button on the microcontroller
8. Run the MATLAB code
9. Press Enter on the MATLAB command window to begin communication with the microcontroller
10. Press PJ0 to begin scanning
11. When scans are completed, a 3D Plot will appear on the PC

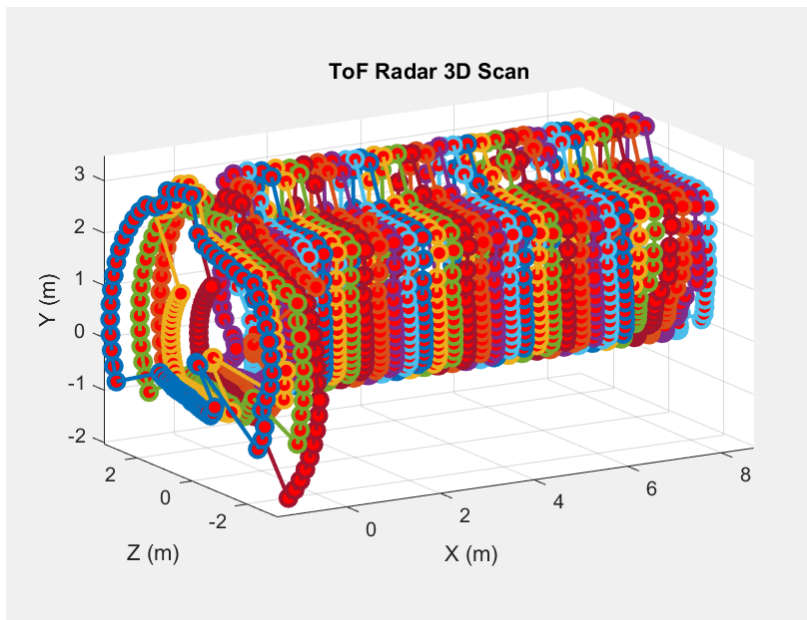
**Expected Output**  
**Figure 2: Scanned Hall**



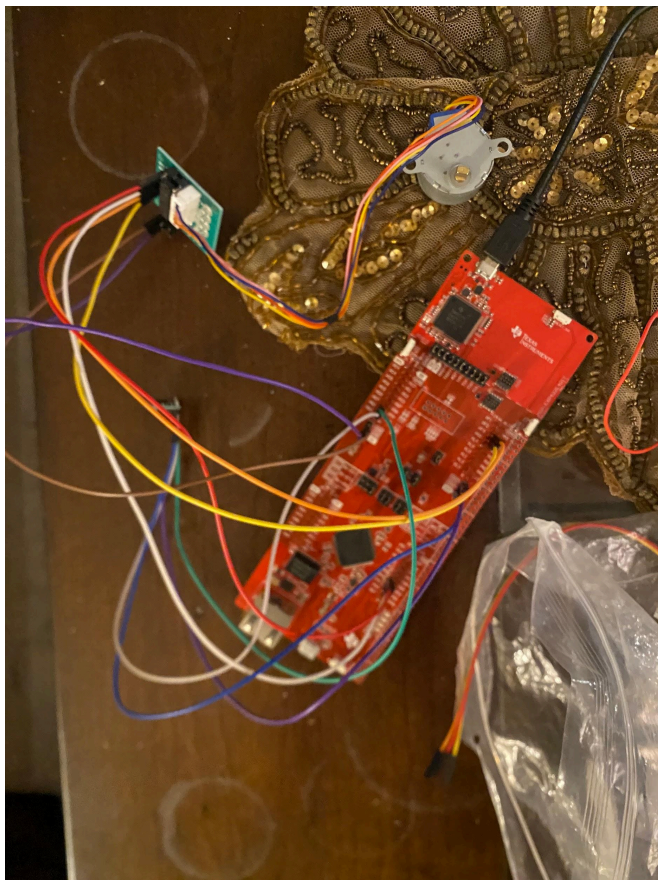
**Figure 3: 3D Plot Front View**



**Figure 4: 3D Plot Side View**



**Figure 5: Example of Circuit Wiring**



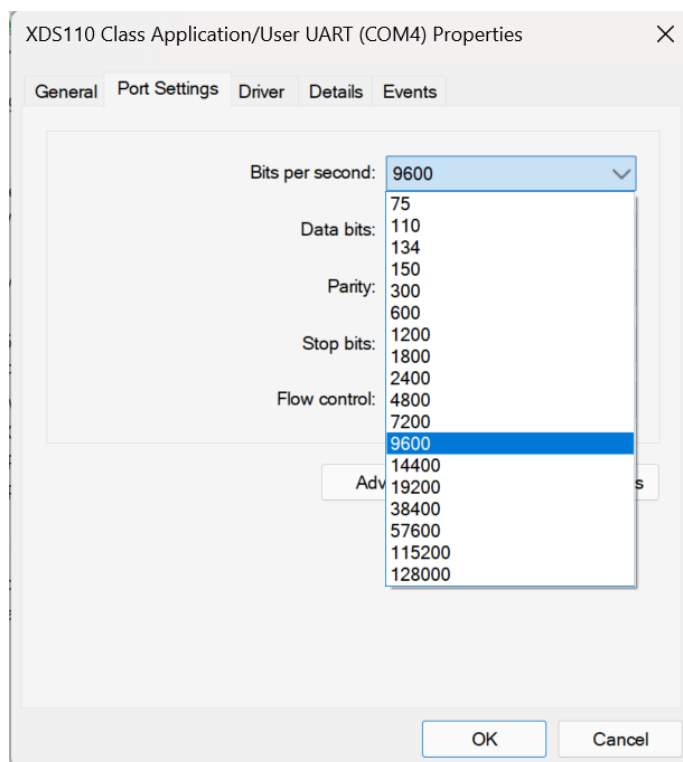
An example of the expected output can be seen in **Figures 3** and **4**, where the hall in **Figure 2** was scanned. From the 3D plot, the shape of the hall can be observed, especially some noticeable features such as the 2 different height levels in the ceiling. It can be seen in the hall and the plot that one part of the hall has a higher ceiling than the other. The second noticeable feature can be seen clearly in **Figure 3**, where the desks in the hall are scanned. Finally, in the beginning, the wall is wider than the rest of the hall, which can be seen in the plots. Finally, the wiring of the circuit can be seen in **Figure 5**.

## Limitations

The microcontroller uses single-precision floating-point arithmetic through its Floating Point Unit (FPU), which leads to rounding errors. Trigonometric calculations often lead to long decimal values, so early rounding can significantly affect the data and ultimately the plot. As a result, the trig calculations were done in MATLAB, as it can do 64-bit calculations, meaning it is more accurate. The calculations done in the trigonometric functions often lead to long decimal values, and when rounding inaccuracies occur, they can affect the final plots generated. Despite using MATLAB for the trig calculations, it still depends on the initial measurements from the microcontroller, which uses 32 bits for the measurement, making some of the data inaccurate. As a result, the final plot has inaccuracies.

The ToF sensor provides measurements with a resolution of 1mm. The maximum quantisation error can be calculated by  $\text{resolution}/2$ , which for this case is  $1\text{mm}/2 = 0.5\text{mm}$ .

**Figure 6: Maximum Bits Per Second**

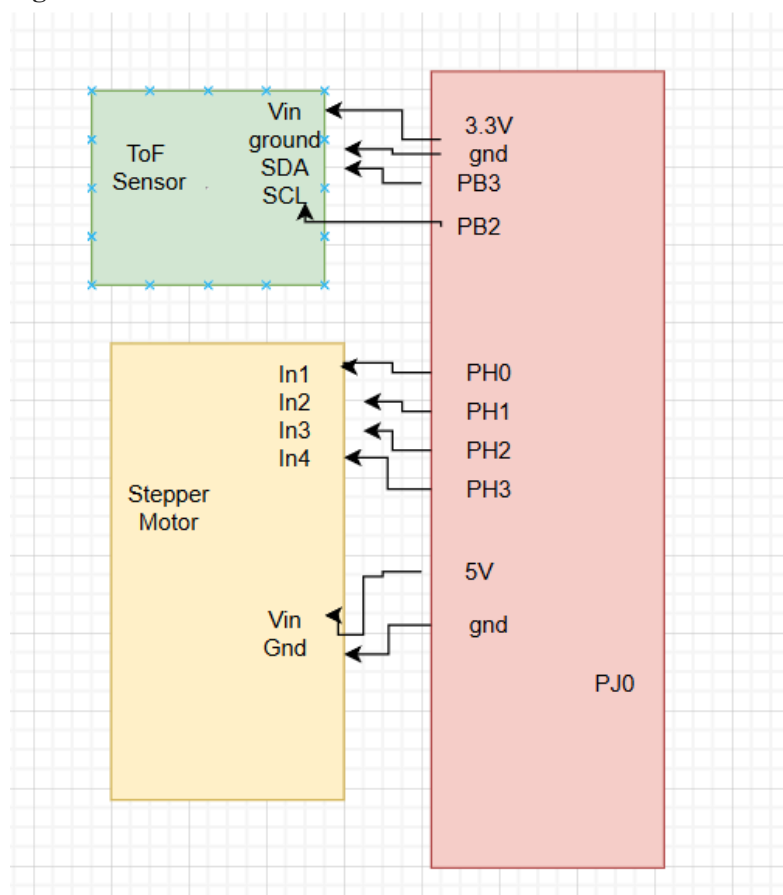


My PC can handle a maximum of 128,000 bps, while the speed in the program is 115200 bps. This speed is used because it is widely supported by most PCs. The maximum speed can be found by opening Device Manager, selecting Ports, then selecting Ports Information, and opening the drop-down menu to see what the listed maximum speed is. This is shown in **Figure 6**.

The communication between the microcontroller and the ToF sensor is done using the I2C protocol. The microcontroller is the master device while the sensor is the slave. The microcontroller retrieves distance measurements as well as range status. The data is then transmitted to MATLAB through UART communication from the microcontroller.

The elements that are responsible for the speed are the ToF measurements time, stepper motor rotation speed and finally UART transmission rate. The ToF sensor requires time for each measurement, which limits how quickly new data can be acquired. The stepper motor also has a delay between each step to ensure reliability and to make sure that there are no missed steps. Finally, the UART communication rate also limits how quickly the data is sent to the computer. The limitations were evaluated by reducing the motor delay and testing until clear instability was seen.

**Figure 7: Circuit Schematic**



Flow Charts

Figure 8: Microcontroller Code Flowchart

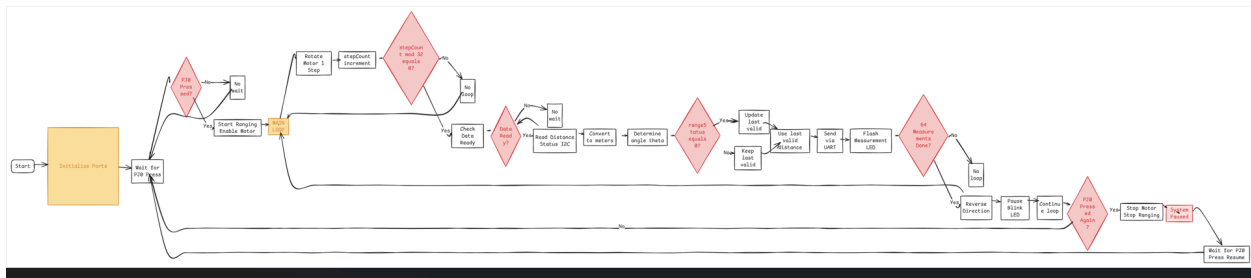
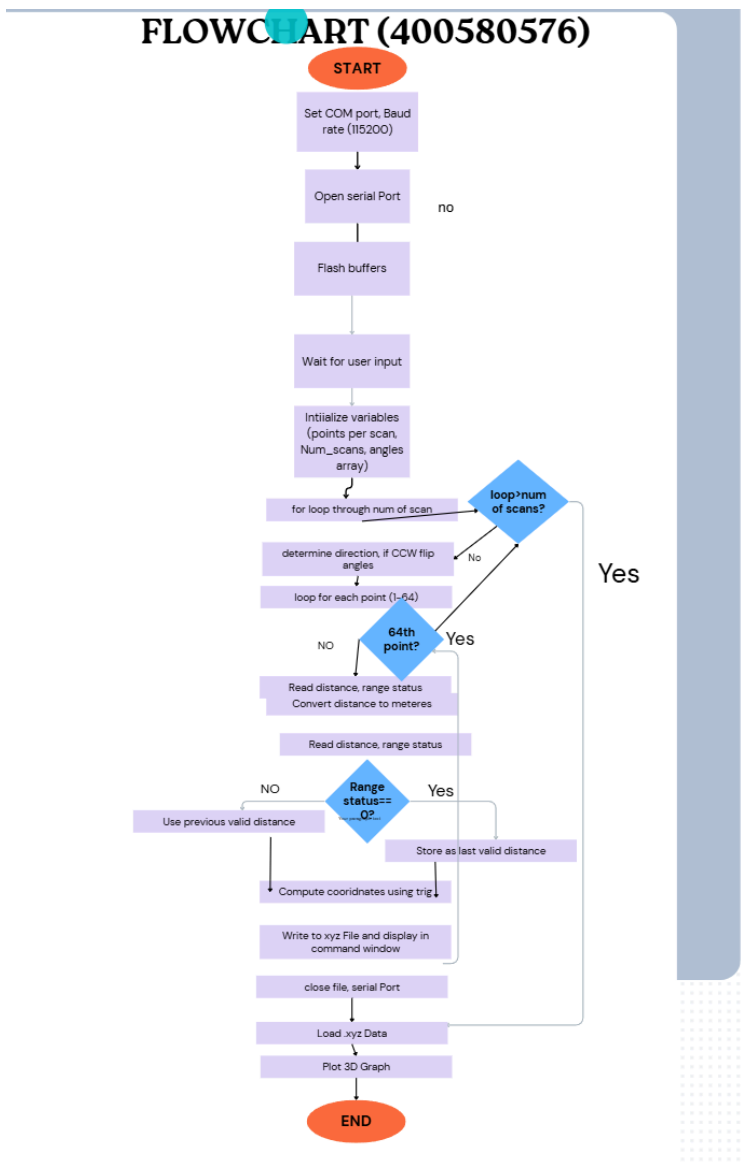


Figure 9: MATLAB Code Flowchart



## References

- [1] Texas Instruments, *MSP432E4 Microcontroller Technical Reference Manual*, 2020.
- [2] STMicroelectronics, *VL53L1X Time-of-Flight Ranging Sensor Datasheet*, Rev. 3, Nov. 2018.
- [3] Velleman, *VMA401 5V Stepper Motor with ULN2003 Driver Board User Manual*, 2018.